

Unit IV Class Activity

1. Study Hours vs Marks — Scatterplot in R

R Program

```
# Create dataset

study_hours <- c(2, 3, 4, 5, 6, 7, 8, 9)
marks <- c(45, 52, 58, 63, 69, 74, 81, 88)
data <- data.frame(study_hours, marks)

# Display dataset

print(data)

# Scatterplot

plot(study_hours, marks,
     main = "Study Hours vs Marks",
     xlab = "Study Hours",
     ylab = "Marks",
     pch = 19)

# Correlation

correlation <- cor(study_hours, marks)

print(correlation)
```

Interpretation

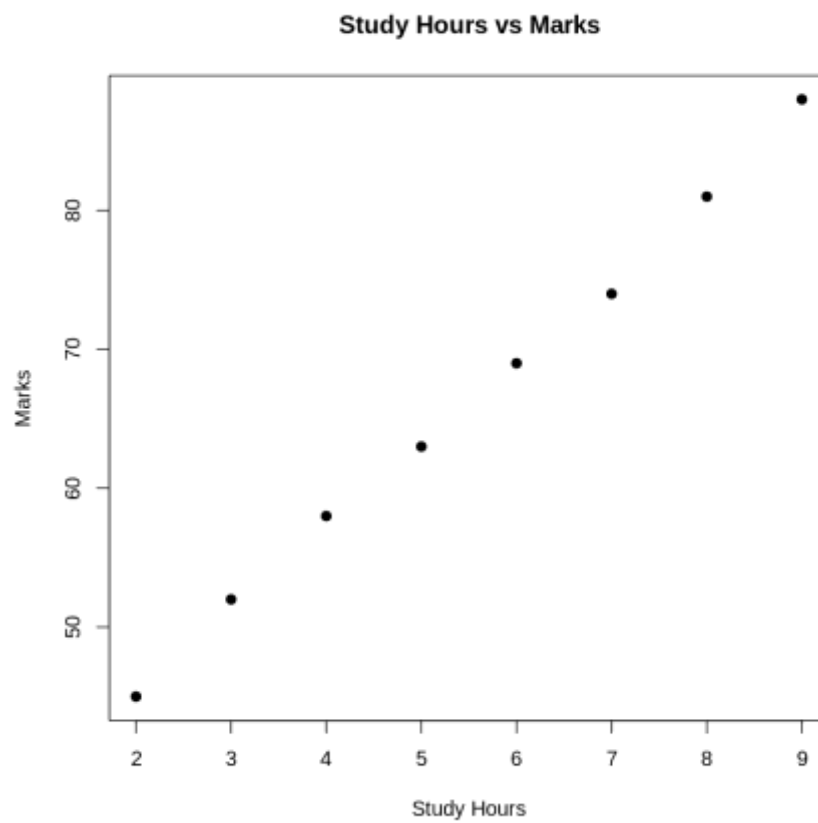
The scatterplot shows an **upward trend**. As study hours increase, students' marks also increase. Therefore, there is a **strong positive correlation** between study hours and marks.

The correlation is approximately **0.998**, indicating a very strong positive relationship.

Output :

	study_hours	marks
1	2	45
2	3	52
3	4	58
4	5	63
5	6	69
6	7	74
7	8	81
8	9	88

[1] 0.998858



2. Age vs Systolic Blood Pressure

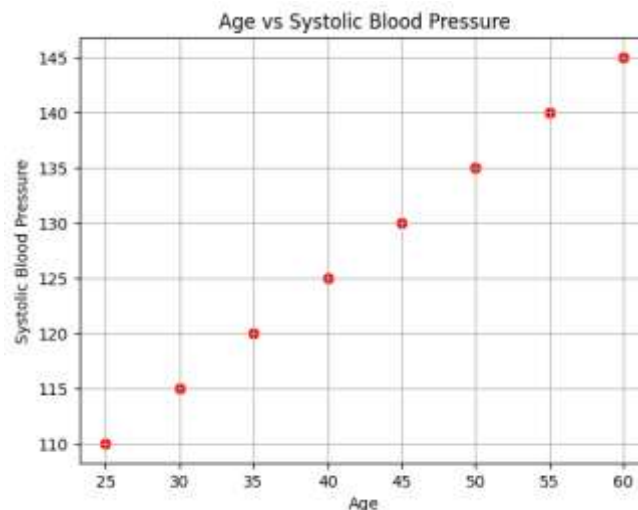
Python Program

```
import matplotlib.pyplot as plt
age = [25, 30, 35, 40, 45, 50, 55, 60]
blood_pressure = [110, 115, 120, 125, 130, 135, 140, 145]
plt.scatter(age, blood_pressure, color="red")
plt.xlabel("Age")
plt.ylabel("Systolic Blood Pressure")
plt.title("Age vs Systolic Blood Pressure")
plt.grid(True)
plt.show()
```

Comment

The scatterplot shows a **positive trend**. Systolic blood pressure tends to increase as patient age increases.

Output



3. Advertising Expenditure vs Monthly Sales

Python Program

```
import matplotlib.pyplot as plt

advertising = [10, 20, 30, 40, 50, 60, 70, 80]
sales = [100, 120, 145, 160, 185, 205, 225, 250]

plt.scatter(advertising, sales)

plt.xlabel("Advertising Expenditure")
plt.ylabel("Monthly Sales")

plt.title("Advertising Expenditure vs Monthly Sales")

plt.grid(True)

plt.show()
```

Interpretation

The scatterplot shows that sales generally increase as advertising expenditure increases. Therefore, there is a **positive relationship** between advertising expenditure and monthly sales.

Output



4. Financial Variables — Correlation Matrix and Correlogram

Since no numerical dataset is provided, the following program creates sample financial data.

Python Program

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Income": [30, 40, 50, 60, 70, 80, 90, 100],
    "Savings": [8, 12, 18, 22, 28, 32, 38, 45],
    "Loan Amount": [15, 20, 25, 30, 35, 40, 45, 50],
    "Credit Score": [600, 620, 640, 660, 680, 700, 720, 740]
}

df = pd.DataFrame(data)

# Correlation matrix
corr = df.corr()
print(corr)

# Correlogram
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Financial Variables Correlation Matrix")
plt.show()
```

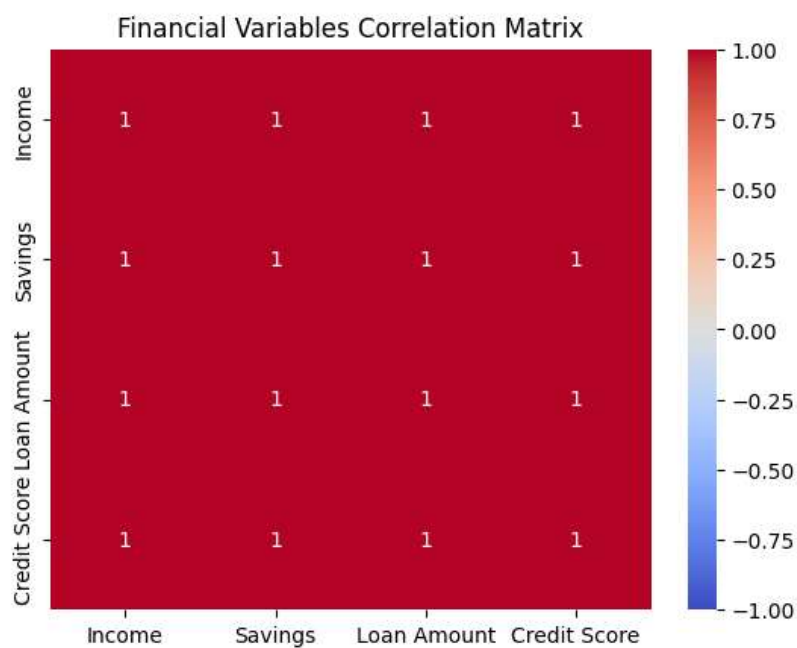
Interpretation

The correlation matrix shows the strength of relationships between the variables.

In this sample data, **Income and Savings** exhibit a strong positive correlation. Income also shows positive relationships with Loan Amount and Credit Score.

Output

Income	Savings	Loan Amount	Credit Score
1.00000	0.99774	1.00000	1.00000
0.99774	1.00000	0.99774	0.99774
1.00000	0.99774	1.00000	1.00000
1.00000	0.99774	1.00000	1.00000



5. Iris Dataset — Correlation Matrix and Heatmap

Python Program

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load Iris dataset

iris = sns.load_dataset("iris")

# Select numerical columns

numeric_data = iris.iloc[:, :4]
```

```

# Correlation matrix
corr = numeric_data.corr()
print(corr)

# Heatmap
sns.heatmap(corr,
             annot=True,
             cmap="coolwarm",
             vmin=-1,
             vmax=1)

plt.title("Iris Dataset Correlation Heatmap")
plt.show()

```

Negative Correlation

For the Iris dataset, the correlation between **sepal length** and **sepal width** is negative.

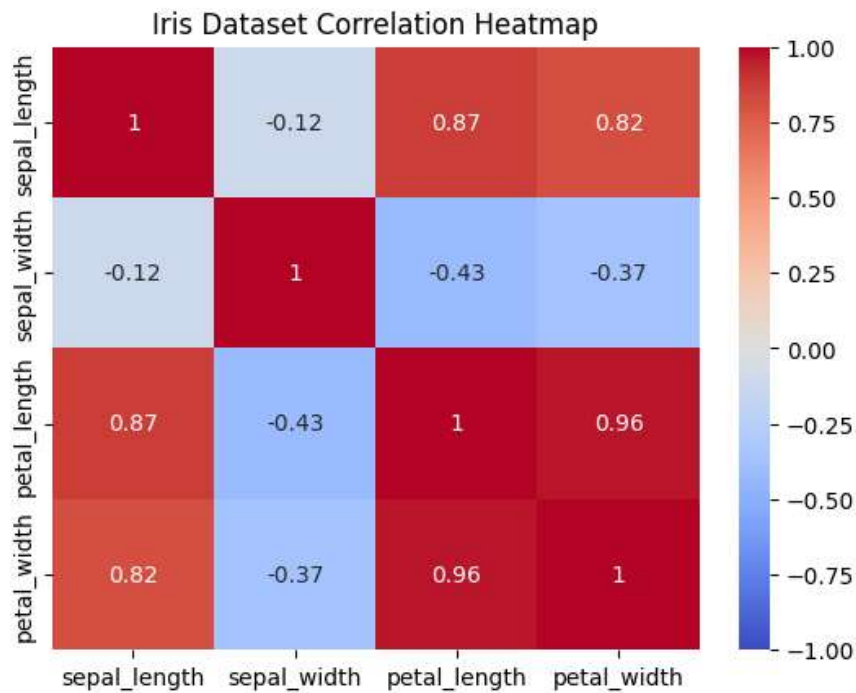
The approximate correlation is:

Sepal Length ↔ Sepal Width = -0.12

Therefore, they have a **weak negative correlation**.

Output

sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000



6. Healthcare Dataset — Correlogram and Multicollinearity

Python Program

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "BMI": [22, 24, 26, 28, 30, 32, 34, 36],
    "Cholesterol": [170, 180, 190, 200, 210, 220, 230, 240],
    "Blood Pressure": [110, 115, 120, 125, 130, 135, 140, 145],
    "Glucose": [80, 90, 100, 110, 120, 130, 140, 150],
    "Heart Rate": [65, 68, 70, 72, 75, 77, 80, 82]
}

df = pd.DataFrame(data)
corr = df.corr()
```

```
print(corr)
sns.heatmap(corr,
             annot=True,
             cmap="coolwarm",
             vmin=-1,
             vmax=1)
plt.title("Healthcare Variables Correlogram")
plt.show()
```

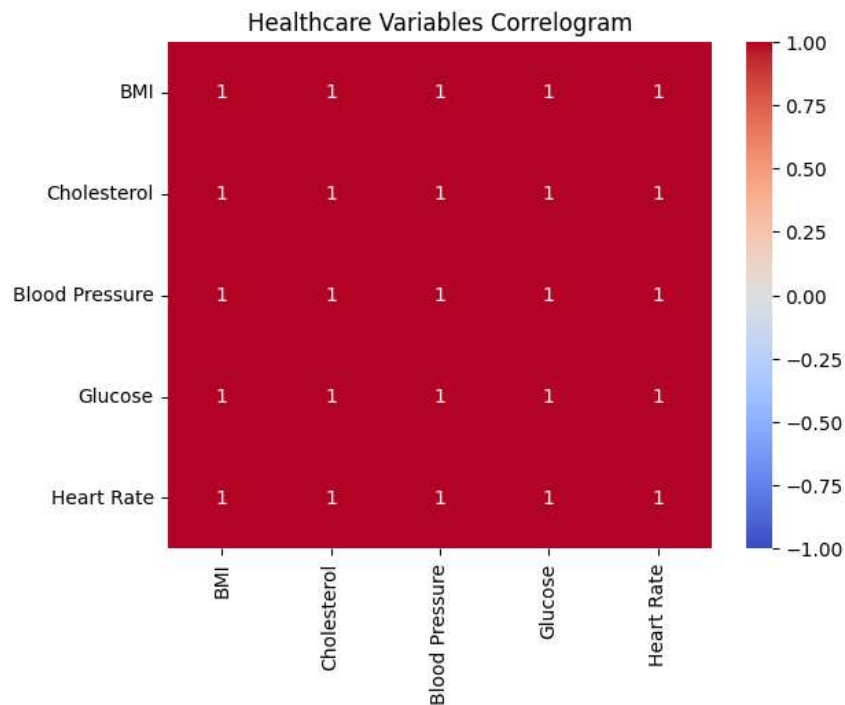
Interpretation

Multicollinearity occurs when two or more independent variables are highly correlated.

In the sample dataset, variables such as **BMI, Cholesterol, Blood Pressure, and Glucose** show strong positive relationships. Very high correlation values, close to **+1**, indicate possible multicollinearity.

Output

BMI	1.000000	1.000000	1.000000	1.000000	0.998813
Cholesterol	1.000000	1.000000	1.000000	1.000000	0.998813
Blood Pressure	1.000000	1.000000	1.000000	1.000000	0.998813
Glucose	1.000000	1.000000	1.000000	1.000000	0.998813
Heart Rate	0.998813	0.998813	0.998813	0.998813	1.000000



7. MRI Dataset — PCA to Two Components

Assume the MRI dataset contains 25 numerical features.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Generate sample MRI data
np.random.seed(42)
X = np.random.rand(100, 25)

# Standardize data
X_scaled = StandardScaler().fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

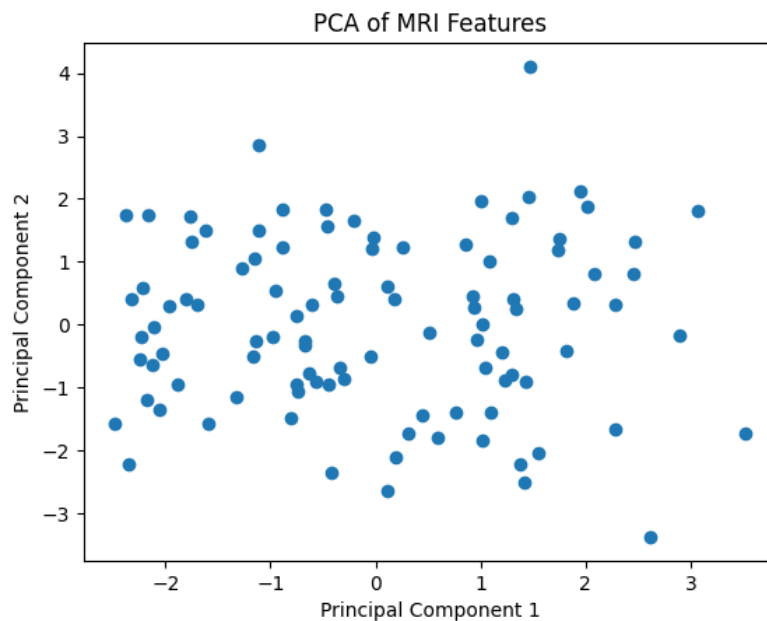
Plot

```
plt.scatter(X_pca[:, 0], X_pca[:, 1])  
plt.xlabel("Principal Component 1")  
plt.ylabel("Principal Component 2")  
plt.title("PCA of MRI Features")  
plt.show()
```

Interpretation

PCA reduces the original **25 dimensions to 2 principal components** while retaining as much important variation as possible.

Output



8. Iris Dataset — PCA and Biplot

Python Program

```
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.datasets import load_iris
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load dataset

iris = load_iris()
X = iris.data
y = iris.target

# Standardize

X_scaled = StandardScaler().fit_transform(X)

# PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Display summary

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
print("\nTotal Variance Explained:")
print(sum(pca.explained_variance_ratio_))

# Plot

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Biplot - Iris Dataset")

# Feature vectors

features = iris.feature_names
for i, feature in enumerate(features):
    plt.arrow(0, 0,
              pca.components_[0, i] * 2,
```

```
pca.components_[1, i] * 2,  
head_width=0.05)  
plt.text(pca.components_[0, i] * 2.1,  
pca.components_[1, i] * 2.1,  
feature)  
plt.show()
```

Interpretation

Principal Component 1 (PC1) explains the highest amount of variance because PCA orders the components according to the amount of variance they explain.

For the Iris dataset, PC1 generally explains about **73% of the variance**, while PC2 explains about **23%**.

Thus, the first two components explain approximately **96% of the total variance**.

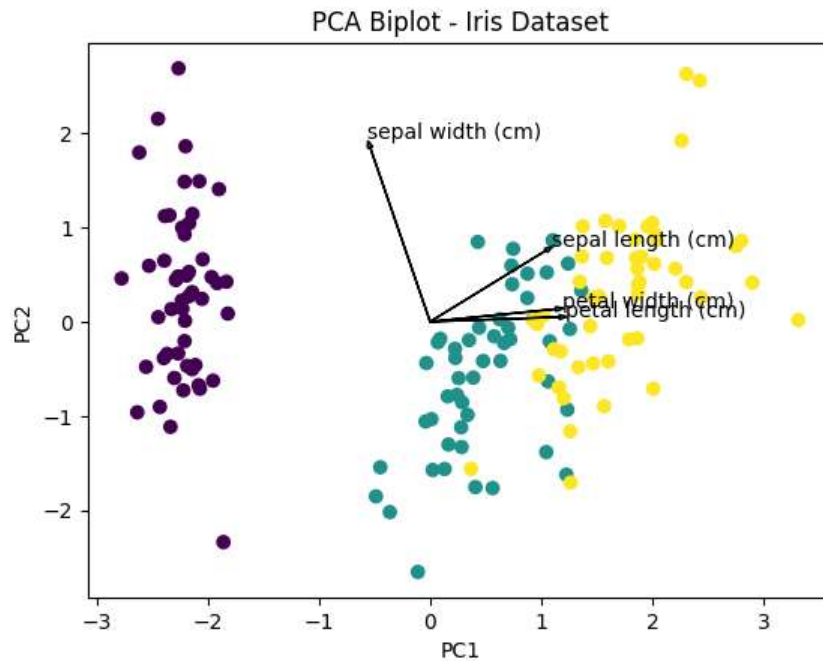
Output

Explained Variance Ratio:

```
[0.72962445 0.22850762]
```

Total Variance Explained:

```
0.9581320720000166
```



9. Automobile Dataset — PCA

Assume the automobile dataset contains 15 numerical specifications.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Sample automobile dataset
np.random.seed(10)
X = np.random.rand(100, 15)

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

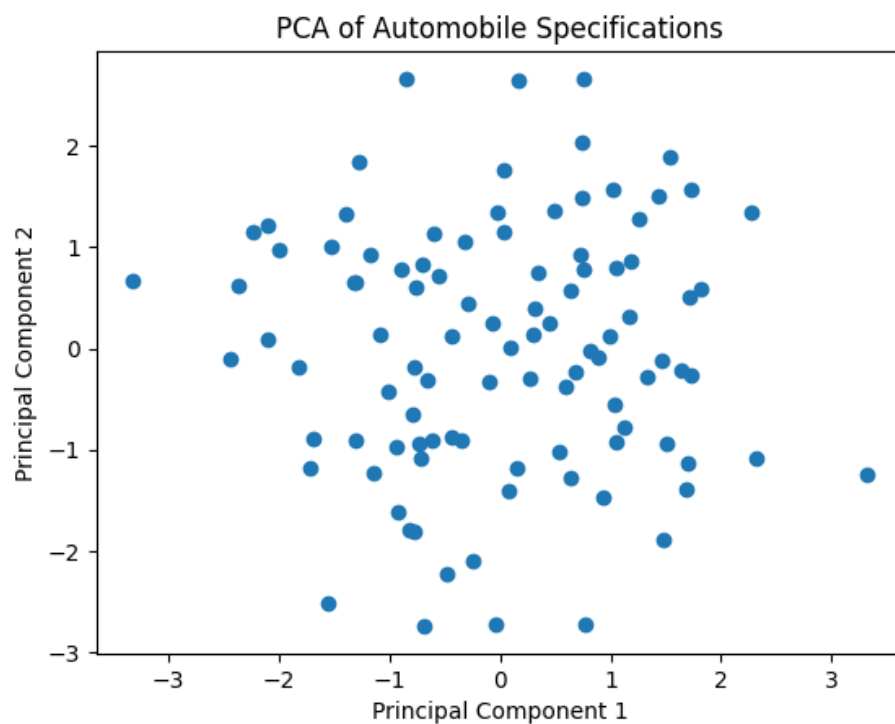
```
# Visualization
```

```
plt.scatter(X_pca[:, 0], X_pca[:, 1])  
plt.xlabel("Principal Component 1")  
plt.ylabel("Principal Component 2")  
plt.title("PCA of Automobile Specifications")  
plt.show()
```

Interpretation

The original **15 automobile specifications** are reduced to two principal components. The plot makes it easier to identify patterns and groups among automobiles.

Output



10. Facial Recognition — t-SNE

A facial recognition dataset contains 128-dimensional embeddings.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
# Generate sample 128-dimensional facial embeddings
np.random.seed(42)
X = np.random.rand(100, 128)
# Apply t-SNE
tsne = TSNE(n_components=2,
            random_state=42,
            perplexity=30)
X_tsne = tsne.fit_transform(X)
# Plot
plt.scatter(X_tsne[:, 0], X_tsne[:, 1])
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("t-SNE Visualization of Facial Embeddings")
plt.show()
```

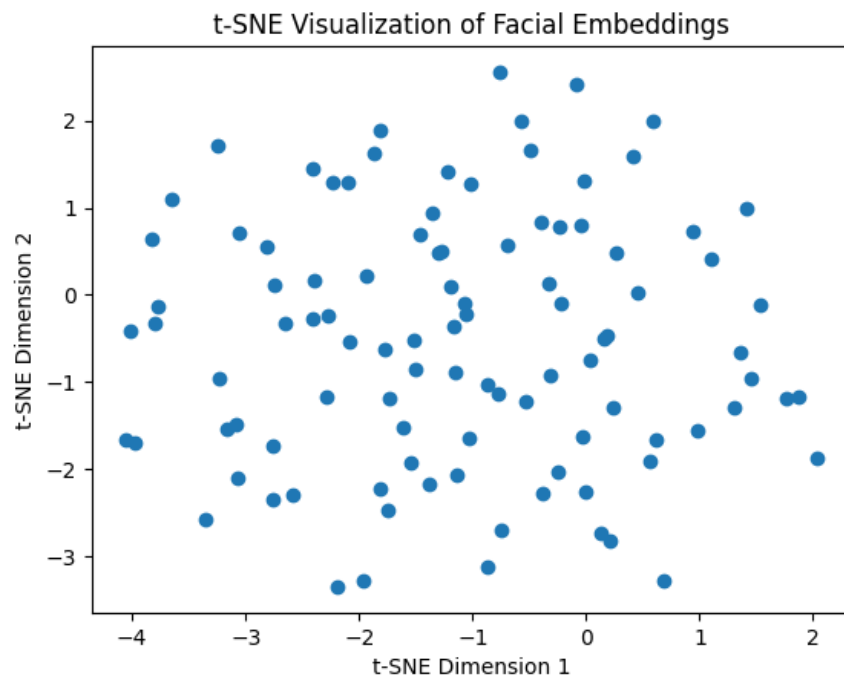
Why t-SNE instead of PCA?

PCA is a linear dimensionality-reduction technique. It mainly preserves global variance.

t-SNE is a nonlinear technique that is particularly useful for visualizing **local similarities and clusters** in high-dimensional data.

For facial embeddings, similar faces may form groups. Therefore, t-SNE can provide a more visually meaningful representation of these groups.

Output



11. Iris Dataset — t-SNE

Python Program

```
import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.manifold import TSNE

# Load Iris dataset

iris = sns.load_dataset("iris")

X = iris.iloc[:, :4]

y = iris["species"]

# Apply t-SNE

tsne = TSNE(n_components=2, random_state=42)

X_tsne = tsne.fit_transform(X)

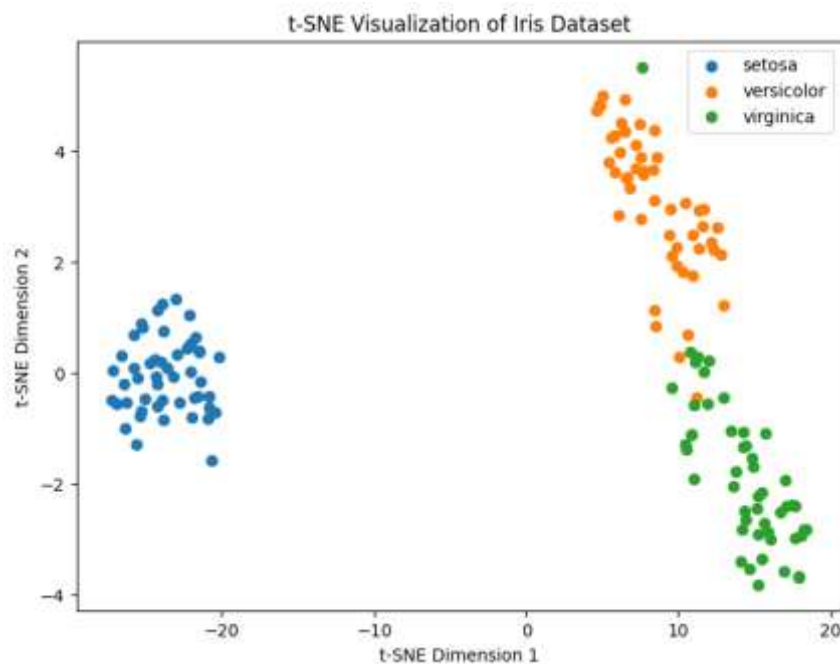
# Plot
```

```
plt.figure(figsize=(8, 6))  
for species in y.unique():  
    index = y == species  
    plt.scatter(X_tsne[index, 0],  
                X_tsne[index, 1],  
                label=species)  
plt.xlabel("t-SNE Dimension 1")  
plt.ylabel("t-SNE Dimension 2")  
plt.title("t-SNE Visualization of Iris Dataset")  
plt.legend()  
plt.show()
```

Interpretation

The t-SNE plot generally shows that the Iris species form separate groups. **Setosa** is usually clearly separated, while **Versicolor** and **Virginica** can have some overlap.

Output



12. Customer Segmentation — t-SNE

The dataset contains 40 behavioral variables.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE

# Sample customer dataset

np.random.seed(42)

X = np.random.rand(200, 40)

# Standardize

X_scaled = StandardScaler().fit_transform(X)

# t-SNE

tsne = TSNE(n_components=2,
            random_state=42,
            perplexity=30)

X_tsne = tsne.fit_transform(X_scaled)

# Plot

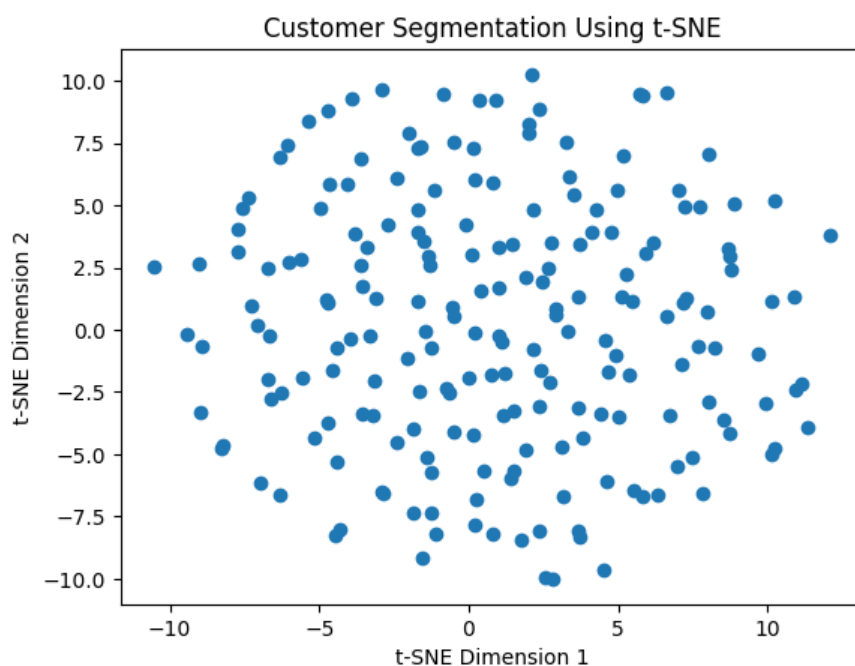
plt.scatter(X_tsne[:, 0], X_tsne[:, 1])
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.title("Customer Segmentation Using t-SNE")
plt.show()
```

Interpretation

t-SNE maps the 40 behavioral variables into two dimensions. If clearly separated groups appear in the plot, they may represent different **customer segments**.

If the points overlap heavily, the dataset does not show clearly separated groups using the selected t-SNE settings.

Output



13. Medical Image Dataset — UMAP vs PCA

The dataset contains 100 extracted features.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import umap.umap_ as umap
```

```
# Generate sample medical image data
np.random.seed(42)
X = np.random.rand(150, 100)

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# UMAP
reducer = umap.UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X_scaled)

# PCA plot
plt.figure()
plt.scatter(X_pca[:, 0], X_pca[:, 1])
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA - Medical Image Dataset")
plt.show()

# UMAP plot
plt.figure()
plt.scatter(X_umap[:, 0], X_umap[:, 1])
plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.title("UMAP - Medical Image Dataset")
plt.show()
```

PCA vs UMAP

PCA	UMAP
Linear technique	Nonlinear technique
Preserves variance	Preserves local structure
Faster and simple	Useful for complex structures
Good for general dimensionality reduction Good for cluster visualization	

UMAP may reveal clusters and nonlinear patterns that PCA cannot clearly show.

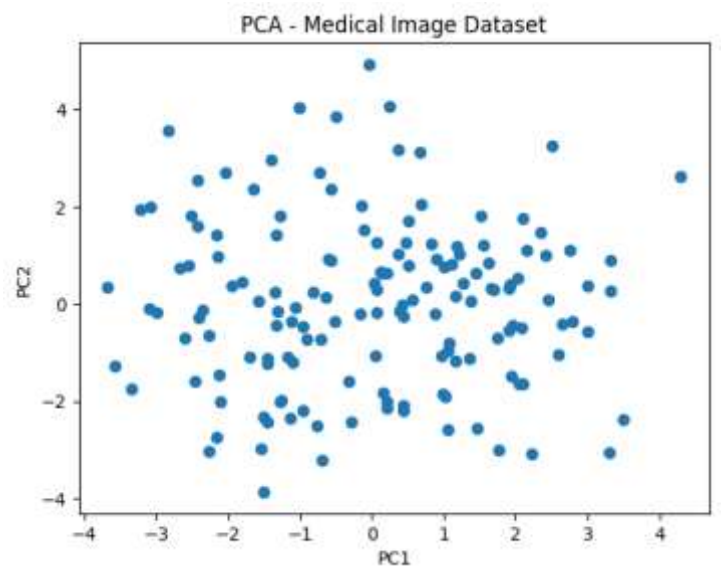
Installation

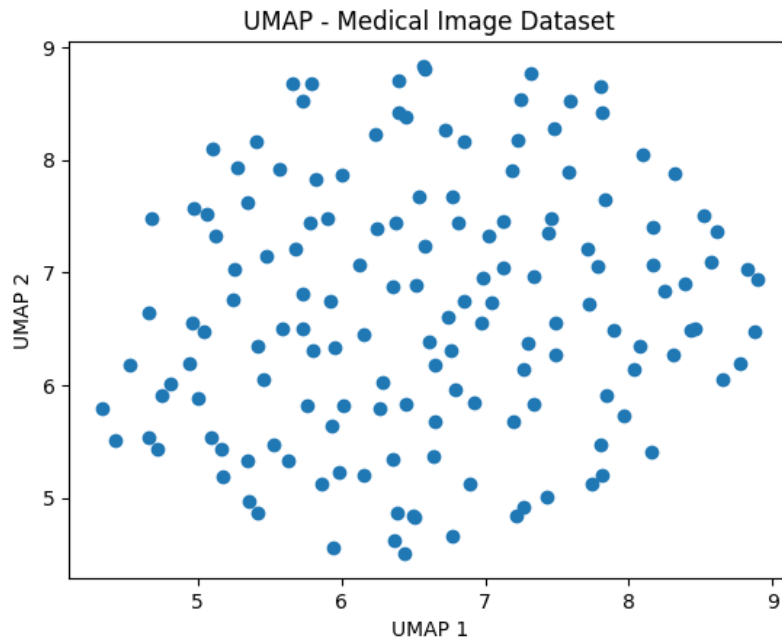
If UMAP is not installed:

Bash

pip install umap-learn

Output





14. Iris Dataset — UMAP

Python Program

```
import matplotlib.pyplot as plt

import seaborn as sns

import umap.umap_ as umap

# Load Iris dataset

iris = sns.load_dataset("iris")

X = iris.iloc[:, :4]

y = iris["species"]

# Apply UMAP

reducer = umap.UMAP(n_components=2, random_state=42)

X_umap = reducer.fit_transform(X)

# Plot

plt.figure(figsize=(8, 6))
```

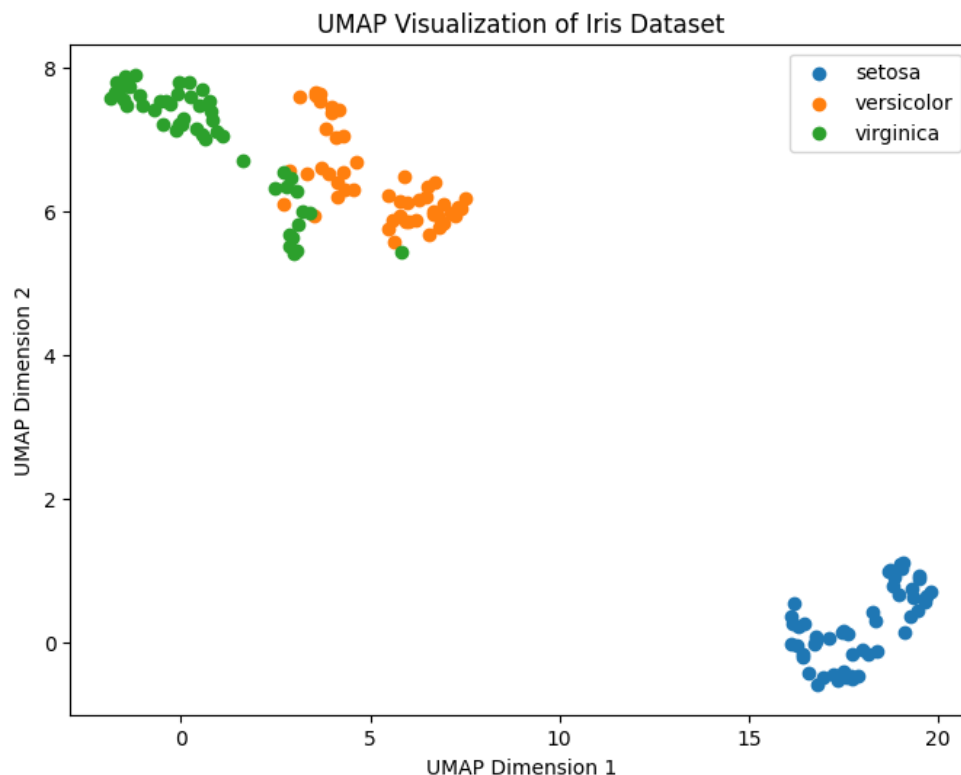
```
for species in y.unique():
    index = y == species
    plt.scatter(X_umap[index, 0],
                X_umap[index, 1],
                label=species)

plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.title("UMAP Visualization of Iris Dataset")
plt.legend()
plt.show()
```

Interpretation

UMAP reduces the four Iris features into two dimensions. Different species generally form separate groups, making it easier to visually identify the structure of the dataset.

Output



15. E-Commerce Customer Behavior — UMAP

The dataset contains 60 purchasing-behavior attributes.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
import umap.umap_ as umap

# Generate sample e-commerce dataset

np.random.seed(42)

X = np.random.rand(200, 60)

# Standardize

X_scaled = StandardScaler().fit_transform(X)

# UMAP

reducer = umap.UMAP(n_components=2,
                    random_state=42)

X_umap = reducer.fit_transform(X_scaled)

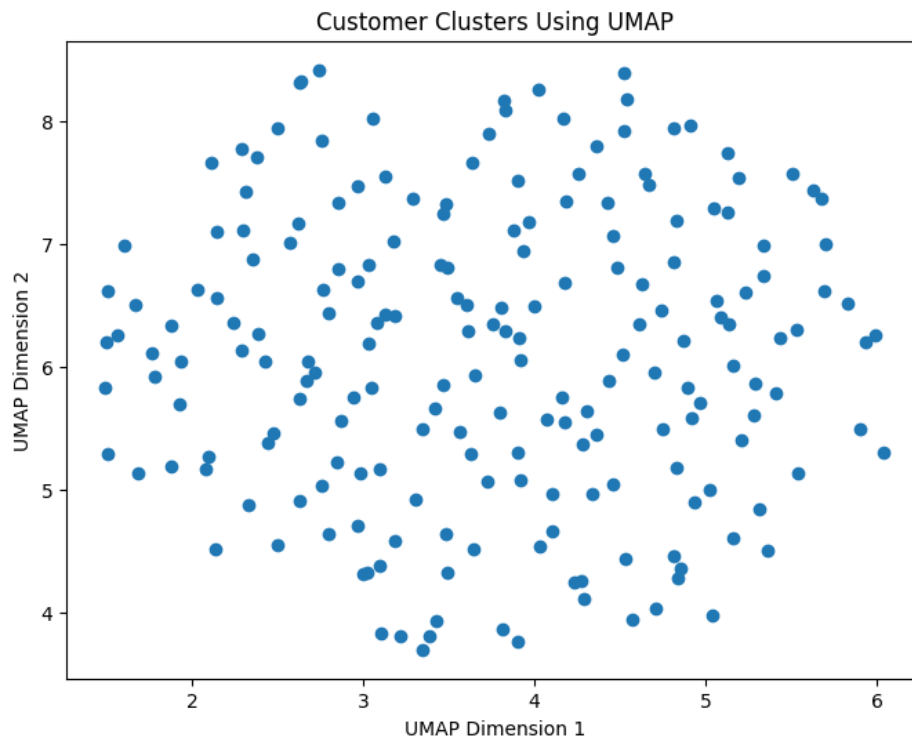
# Plot

plt.figure(figsize=(8, 6))
plt.scatter(X_umap[:, 0],
           X_umap[:, 1])
plt.xlabel("UMAP Dimension 1")
plt.ylabel("UMAP Dimension 2")
plt.title("Customer Clusters Using UMAP")
plt.show()
```

Interpretation

UMAP reduces the **60 customer attributes into two dimensions**. Customers located close together have similar purchasing behavior. Distinct groups in the visualization can be interpreted as potential **customer clusters or segments**.

Output



Overall Conclusion

These questions demonstrate several important **data visualization and dimensionality-reduction techniques**:

Q.No	Technique	Purpose
1	Scatterplot	Study Hours vs Marks
2	Scatterplot	Age vs Blood Pressure
3	Scatterplot	Advertising vs Sales

Q.No	Technique	Purpose
4	Correlation Matrix	Financial relationships
5	Heatmap	Iris correlations
6	Correlogram	Healthcare multicollinearity
7	PCA	MRI dimensionality reduction
8	PCA + Biplot	Iris dimensionality reduction
9	PCA	Automobile features
10	t-SNE	Facial embeddings
11	t-SNE	Iris clusters
12	t-SNE	Customer groups
13	UMAP + PCA	Medical image features
14	UMAP	Iris clusters
15	UMAP	E-commerce customer clusters